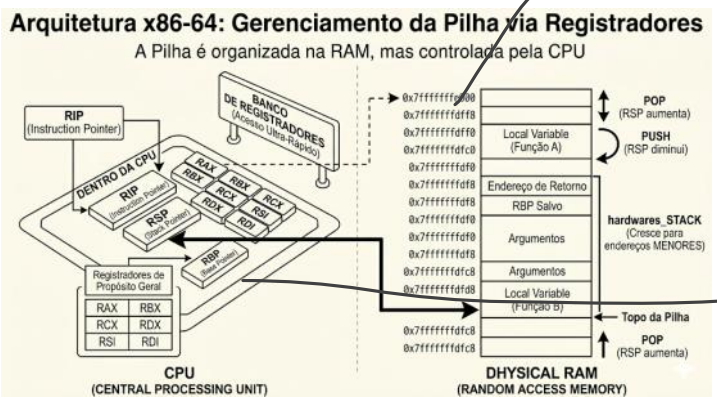
Registradores

Tuesday, April 21, 2026 10:13 AM



Indice do Array

Endereço Físico

Usa registradores para acessar o endereço de memória do dado que quer acessar na RAM

- Processador calcula o endereço físico usando somas e multiplicações.
- Entre o endereço que a CPU calcula (Somas e Multiplicações) e o chip de RAM, existe um componente de hardware chamado **MMU (Memory Management Unit)**.
- A **MMU** intercepta esse valor e consulta uma "tabela de tradução" (Page Tables) criada pelo Sistema Operacional
.
- A **MMU** descobre que o endereço lógico 0x1000 na verdade corresponde ao endereço físico 0x5A000 na RAM real.

- **RSP (Stack Pointer)**: Mantém o endereço do topo da pilha e deve ser alinhado em **16 bytes** antes de chamadas.
- **RBP (Base Pointer)**: Mantém o endereço base da pilha.
- **RIP (Instruction Pointer)**: Aponta para a próxima instrução a ser executada.

Endereçamento

Tuesday, April 21, 2026 10:43 AM

Existem algumas regras que diferenciam o uso de endereços de memória, quando se realiza cálculos aritméticos, que são regras de acordo com o tamanho dos endereços utilizados

- Em resumo, alguns registradores podem ser utilizados, e outros não

Endereçamento - 16 bits

Imagine que a CPU precisa de uma fórmula para encontrar um dado na RAM, mas essa fórmula tem regras rígidas.

Aqui está a explicação simplificada de cada ponto:

1. O que é o byte ModR/M?

Pense no **ModR/M** como o "formulário de instruções" que vem junto com o comando (como o MOV).

2. O formato: REG + REG + DESLOCAMENTO

Diferente de hoje, onde podemos somar quase qualquer coisa, em 16 bits a fórmula era uma combinação fixa de no máximo três partes:

1. **Um registrador de Base**
2. **Um registrador de Índice**
3. **Um valor fixo (Deslocamento)**

3. Quem faz o quê? (Base vs. Índice)

Nessa época, os registradores tinham funções "especializadas" e você não podia trocar os papéis:

- **Bases (BX e BP):** São os "pontos de partida". Geralmente apontam para o início de uma lista ou de uma função.
- **Índices (SI e DI):** São os "contadores". Eles dizem quantos passos à frente da base você deve olhar (úteis para percorrer strings ou arrays).

4. A Regra de Soma (O que NÃO pode)

Esta é a parte que costuma confundir. O hardware de 16 bits era simples e seus circuitos de soma eram limitados. Por isso:

- **Não pode Base + Base:** Você não pode fazer [BX + BP]. A CPU só tem um "encaixe" para base.
- **Não pode Índice + Índice:** Você não pode fazer [SI + DI]. Só há um "encaixe" para índice.

Como ficava na prática:

- MOV AX, [BX + SI + 10] → **OK!** (Base + Índice + Deslocamento).
- MOV AX, [BX + BP] → **ERRO!** (Duas bases, a CPU "se perde").
- MOV AX, [SI + DI] → **ERRO!** (Dois índices).

Endereçamento - 32 bits

1. O Fator de Escala (A grande novidade)

Agora o processador consegue fazer uma multiplicação "de graça" (em um único ciclo) dentro do endereçamento.

- **Os números mágicos (1, 2, 4, 8):** Eles não são aleatórios. Eles correspondem aos tamanhos padrão de tipos de dados em programação:
- **1:** char ou byte
- **2:** short ou word
- **4:** int ou float (ou endereços de 32 bits)
- **8:** long long, double (ou endereços de 64 bits)

2. Liberdade de Registradores

- **Base:** Qualquer registrador de propósito geral (EAX, EBX, ECX, EDX, EBP, ESP, ESI, EDI) pode ser a base.
- **Índice:** Qualquer um **exceto o ESP**.
- *Por que o ESP não pode ser índice?* Porque o ESP (Stack Pointer) é "sagrado". Ele é gerenciado pelo hardware para a pilha. Usá-lo como um índice multiplicável em um cálculo de array causaria uma confusão imensa na lógica do processador.

3. Deslocamento (8 ou 32 bits)

O valor fixo que você soma ao final pode ser curto (8 bits, para economizar espaço no código) ou longo (32 bits, permitindo saltar para qualquer lugar em um espaço de 4 GB).

Exemplo de "Antes vs. Depois"

Imagine que você tem um array de **inteiros** (cada um com 4 bytes) e quer acessar o elemento na posição guardada em **ECX**.

- **Em 16 bits (Impossível fazer direto):** Você teria que multiplicar o índice por 4 manualmente usando SHL ou MUL e depois colocar em SI/DI para só então acessar a memória.
- **Em 32 bits:**

```
mov eax, [ebx + ecx * 4]
```

- ebx = Início do array (Base)
- ecx = Qual elemento eu quero (Índice)
- 4 = Tamanho de cada inteiro (Escala)

Endereçamento - 64 bits

1. A Regra do "Não Mesclar"

Essa é a principal armadilha em provas. Em 64 bits, você precisa de consistência:

- **Válido:** [RAX + RBX*2] (Tudo 64 bits)
- **Válido:** [EAX + EBX*2] (Tudo 32 bits — embora menos comum em código 64-bit puro)
- **INVÁLIDO:** [RAX + EBX*2] (Tentou somar 64 com 32).

O processador não consegue lidar com tamanhos de barramento diferentes dentro da mesma conta de endereço.

2. Os Novos Recrutas (R8 a R15)

Diferente das gerações passadas, onde tínhamos poucos registradores, agora temos 8 novos registradores de propósito geral (R8, R9, R10...). Eles são totalmente flexíveis: podem ser usados tanto como **base** quanto como **índice** (algo que o ESP ainda não pode fazer).

3. Endereçamento Relativo ao RIP (RIP-Relative Addressing)

Esta é a maior mudança estratégica de arquitetura.

- **Como era antes:** Para acessar uma variável global, o programa usava um endereço fixo (ex: 0x401000). Se o SO carregasse o programa em um lugar diferente da memória, tudo quebrava.
- **Como é agora:** O processador consegue calcular o endereço baseado em "onde eu estou agora" (**RIP**).
- **A fórmula:** Enderec\c o=RIP+Deslocamento(32bits).
- **Vantagem:** Isso permite a criação de **Código Independente de Posição (PIC)**, que é essencial para a segurança moderna (como o ASLR, que "embaralha" os endereços para evitar hackers).

Registradores Propósito Geral

Tuesday, April 21, 2026 11:34 AM

Os registradores de propósito geral (ou **GPRs** - *General Purpose Registers*) são as "áreas de trabalho" mais rápidas da CPU. Eles são usados para armazenar dados temporários, realizar cálculos aritméticos e manipular endereços de memória.

1. Os Oito Registradores Clássicos

Embora hoje quase todos façam qualquer tarefa, eles mantêm nomes baseados em suas funções originais:

- **RAX (Accumulator):** Usado em operações aritméticas e para armazenar o **valor de retorno** de funções.
- **RBX (Base):** Frequentemente usado como ponteiro para dados no segmento de memória.
- **RCX (Counter):** Usado como contador em loops e operações de repetição de strings.
- **RDX (Data):** Usado em operações de multiplicação/divisão e para entrada/saída (I/O).
- **RSI (Source Index):** Ponteiro de origem para operações de cópia de strings/memória.
- **RDI (Destination Index):** Ponteiro de destino para operações de cópia de strings/memória.
- **RBP (Base Pointer):** Aponta para a base do "stack frame" (ajuda a organizar variáveis locais).
- **RSP (Stack Pointer):** Aponta para o topo atual da pilha (Stack). **Atenção:** Embora seja de propósito geral, mexer nele manualmente sem cuidado quebra o programa.

2. Os Novos Registradores (R8 - R15)

Introduzidos com a arquitetura de 64 bits:

- **R8, R9, R10, R11, R12, R13, R14, R15.**
- Eles não possuem funções "especiais" históricas; são usados livremente pelo compilador para cálculos e armazenamento.

3. Anatomia dos Tamanhos (Exemplo com RAX)

Um ponto que aparece nos seus slides é como esses registradores se dividem para acessar menos bits. Veja a hierarquia do acumulador:

- **RAX:** 64 bits (O registrador inteiro).
- **EAX:** 32 bits inferiores.
- **AX:** 16 bits inferiores.
- **AL:** 8 bits inferiores.
- **AH:** 8 bits superiores (apenas dentro dos 16 bits de AX).

4. Uso na Passagem de Argumentos (Linux/ABI)

Em sistemas modernos de 64 bits, os primeiros argumentos que você passa para uma função não vão para a RAM, eles vão direto para estes registradores nesta ordem:

1. **RDI** (1º argumento)
2. **RSI** (2º argumento)
3. **RDX** (3º argumento)
4. **RCX** (4º argumento)
5. **R8** (5º argumento)
6. **R9** (6º argumento)

Imporatente: mov rdi, 1, o programa provavelmente está preparando o primeiro parâmetro para uma chamada de sistema ou função.

LEA

Tuesday, April 21, 2026 11:47 AM

A instrução LEA (Load Effective Address) calcula, e armazena em um registrador, o endereço efetivo de um operando de memória, sem acessar diretamente essa memória

- LEA calcula o endereço efetivo de um operando de memória
- O resultado do cálculo é armazenado em um registrador
- LEA não acessa a memória, apenas realiza o cálculo do endereço

- Instrução utilizada para operações aritméticas simples em endereços

```
_start:
    mov rax, 5
    mov rbx, 10
    lea eax, [rbx + rax * 2 + 5]
```

O Diferencial da LEA: Se usasse instruções normais como ADD e MUL, precisaria de vários ciclos de processamento. Com a LEA, a CPU faz a **multiplicação e as duas somas em um único ciclo de instrução**, pois usa o circuito que já existe para calcular posições de memória.

Importante: O resultado final (**25**) é guardado em **EAX** (a parte de 32 bits de RAX), e o processador **não acessa a memória RAM**; ele apenas guarda o resultado do cálculo no registrador de destino.

1. _start:

É um **rótulo (label)** que indica o ponto de entrada do programa. É por onde o Sistema Operacional começa a execução do código.

2. mov rax, 5 e mov rbx, 10

Estas são instruções de atribuição simples:

- O registrador **RAX** recebe o valor **5**.
- O registrador **RBX** recebe o valor **10**. Neste momento, ambos os valores estão dentro da CPU, prontos para uso.

3. lea eax, [rbx + rax * 2 + 5]

Esta é a parte mais interessante. A instrução **LEA** significa *Load Effective Address* (Carregar Endereço Efetivo). Embora tenha sido criada para calcular endereços de memória, ela é frequentemente usada como um "atalho matemático" super rápido.

O processador utiliza o seu hardware de cálculo de endereços para resolver a expressão dentro dos colchetes:

- **Base (rbx):** 10
- **Índice (rax):** 5
- **Escala (* 2):** Multiplica o índice por 2.
- **Deslocamento (5):** Um valor constante somado ao final.

O Cálculo Realizado:

$10+(5\times2)+5=25$

Resumo do Estado Final:

- **RAX/EAX:** Passa a valer **25**.
- **RBX:** Continua valendo **10**.

MOV

Tuesday, April 21, 2026 12:05 PM

A instrução **MOV** é a operação mais fundamental da linguagem Assembly. Apesar do nome sugerir "mover", o que ela realmente faz é uma **cópia**. O dado original permanece onde estava, e uma cópia idêntica é colocada no destino.

Pense nela como o comando "Atribuir" em linguagens de alto nível (como x = 5).

1. A Sintaxe Básica

A estrutura padrão segue esta ordem: MOV destino, origem

- **Destino:** Onde o dado será guardado (pode ser um registrador ou um endereço de memória).
- **Origem:** De onde o dado vem (pode ser um valor fixo, um registrador ou um endereço de memória).

2. O que você PODE fazer com o MOV

Existem quatro combinações principais permitidas pelo processador:

1. **Registrador para Registrador:** Copia o valor de um "bolso" da CPU para outro.
   - mov rax, rbx (O que estava em RBX agora está em RAX também).
2. **Valor Imediato para Registrador:** Coloca um número fixo direto em um registrador.
   - mov rax, 10 (RAX passa a valer 10).
3. **Memória para Registrador:** Busca um dado lá na RAM e traz para dentro da CPU.
   - mov rax, [rbx] (Olha o endereço guardado em RBX, vai na RAM, e copia o que achou para RAX).
4. **Registrador para Memória:** Pega um resultado da CPU e guarda em uma "gaveta" da RAM.
   - mov [rbx], rax (Pega o valor de RAX e salva no endereço de memória indicado por RBX).

3. O que você NÃO PODE fazer

O hardware da CPU tem limitações físicas. Duas regras são essenciais:

- **Memória para Memória:** Você nunca pode fazer mov [endereço1], [endereço2] diretamente. A CPU não consegue ler e escrever na RAM ao mesmo tempo em uma única instrução. Você deve primeiro mover para um registrador e depois para o destino.
- **Conflito de Tamanhos:** Você não pode mover um dado de 64 bits para um espaço de 16 bits. Os tamanhos devem ser compatíveis.
- mov ax, rax **(ERRO)**: Tentando colocar 64 bits em 16 bits.

4. O Comportamento dos Bits

Como vimos nos seus slides, o comportamento muda dependendo de qual parte do registrador você usa:

- **Usando EAX (32 bits):** Quando você move algo para o eax, o processador **zera automaticamente** os 32 bits superiores do rax. É uma forma de limpeza automática.

- **Usando AX ou AL (16/8 bits):** Se você mover um valor para al (os 8 bits de baixo), o restante do registrador rax **não é alterado**. O valor antigo que estava lá continua lá, o que pode causar confusão se você não tomar cuidado.

Exemplo Prático de Revisão

```
_start:
   mov rax, 0xFFFFFFFFFFFFFFFF  ; Enche o RAX de '1s'
   mov al, 0x11                 ; Muda só o finalzinho. RAX ainda tem quase tudo 'F'
   mov eax, 0x44444444          ; Coloca 4444 no fundo E ZERA o topo. RAX agora é só
0000000044444444
```

Resumo: O MOV é o "copia e cola" da computação. Ele é a ferramenta que você usa para posicionar os dados nos lugares certos (registradores) antes de fazer contas com eles.

Little endian

Tuesday, April 21, 2026 1:23 PM

O **Little-Endian** é uma forma de organizar como o processador armazena números que ocupam mais de um byte (como números de 16, 32 ou 64 bits) na memória RAM.

De forma simples: em um sistema Little-Endian, o **byte menos significativo** (o "final" do número) é armazenado no **menor endereço** de memória.

1. Analogia Prática

Imagine que você vai escrever o número **1.234**.

- No nosso sistema de escrita normal (Big-Endian), escrevemos o "mil" primeiro: **1, 2, 3, 4**.
- No sistema **Little-Endian**, você escreveria o número "de trás para frente" em blocos: **4, 3, 2, 1**.

2. Exemplo Técnico (Hexadecimal)

Imagine o valor hexadecimal 0x12345678. Ele é composto por 4 bytes: 12, 34, 56 e 78.

Se pedirmos para um processador Intel (que usa Little-Endian) salvar esse valor começando no endereço 0x1000, a memória ficaria assim:

Endereço	Byte	Importância
0x1000	**78**	Byte menos significativo (LSB)
0x1001	**56**	
0x1002	**34**	
0x1003	**12**	Byte mais significativo (MSB)

3. Por que isso existe? (História dos seus slides)

Como mencionado no seu material (slide 22), a Intel adotou isso nos anos 70.

- **Aritmética facilitada:** Quando o processador faz somas, ele começa pelos números da direita (os menos significativos). Ter esse byte logo no primeiro endereço facilitava o design dos circuitos de "vai um" (carry) na época.
- **Compatibilidade:** Uma vez que a Intel definiu isso no processador 8008, todos os sucessores (incluindo o seu i7 ou i9 atual) mantiveram para que programas antigos continuassem funcionando.
-
-

4. Onde isso causa confusão?

A confusão acontece principalmente quando você está olhando a memória diretamente (em um debugger).

- Se você coloca 0x11223344 em um registrador, você vê 11223344.
- Mas se você olhar esse mesmo valor **na RAM**, ele parecerá "bagunçado": 44 33 22 11.

Dica importante: O "embaralhamento" só acontece com os **bytes**. Os bits dentro de cada byte continuam na mesma ordem. É como se você mudasse a ordem dos livros na estante, mas não a ordem das páginas dentro de cada livro.

5. Quando ocorre o "embaralhamento"?

Ele só acontece em operações **Multibyte** (dados de 16, 32 ou 64 bits) que envolvem a **Memória RAM**:

- **Store (Salvar):** Quando você move um valor de um registrador para a memória (mov [mem], rax). A CPU quebra o valor e inverte os bytes na RAM.
- **Load (Carregar):** Quando você busca um valor da memória para um registrador (mov rax, [mem]). A CPU pega os bytes invertidos na RAM e os "desembaralha" para que fiquem na ordem correta dentro do registrador.

Por que isso é importante?

Se você for um programador e salvar o número 0x1234 na memória e depois tentar ler byte a byte, você verá 34 e depois 12. Se você não souber que o sistema é Little-Endian, vai achar que seu dado foi corrompido, quando na verdade é apenas a forma como o processador Intel organiza a "estante" de bytes dele para ser mais eficiente.

Resumo: O processador é como um tradutor: ele fala "humano" (ordem normal) dentro de casa (CPU), mas escreve em "código invertido" quando manda cartas para o depósito (RAM).

RIP

Tuesday, April 21, 2026 1:30 PM

Na arquitetura x86-64, **RIP** é o acrônimo para **Instruction Pointer Register** (Registrador de Ponteiro de Instrução).

Ele é, talvez, o registrador mais importante para o controle do fluxo de um programa, pois ele funciona como o "maestro" ou um "GPS" da CPU.

Aqui está o que você precisa saber sobre ele:

1. A Função Principal

O RIP armazena o **endereço de memória da próxima instrução** que o processador deve executar.

- A CPU funciona em um ciclo infinito de: **Busca** (Fetch) -> **Decodifica** -> **Executa**.
- O RIP diz à CPU exatamente de onde "buscar" a próxima linha de código na RAM.

2. O Comportamento Automático

Normalmente, você não altera o RIP manualmente com um MOV. Ele se atualiza sozinho:

- **Sequencial:** Assim que a CPU busca uma instrução, o RIP é incrementado automaticamente para apontar para a instrução logo abaixo.
- **Desvios:** Quando o código tem um JMP (pulo), um CALL (chamada de função) ou um RET (retorno), o valor do RIP é alterado para o novo endereço, fazendo o programa "saltar" para outra parte da memória.

3. Endereçamento Relativo ao RIP (RIP-Relative)

Como vimos nos seus slides anteriores, no modo de 64 bits, o RIP é usado para calcular endereços de dados:

- lea rax, [rip + 0x200] Isso significa: "pegue o endereço da instrução atual e some 200 bytes". Isso permite que o programa encontre suas variáveis globais não importa em que lugar da RAM o sistema operacional o tenha colocado (isso é a base do **ASLR** - *Address Space Layout Randomization*, uma medida de segurança).

4. Hierarquia de Nomes

Assim como os outros registradores, o nome muda conforme a arquitetura (tamanho dos bits):

- **IP:** 16 bits (8086 original).
- **EIP:** 32 bits (Extended IP).
- **RIP:** 64 bits (Registro atual que você está estudando).

Resumo: Se o processador fosse um leitor de livros, o **RIP** seria o seu **dedo** apontando para a próxima palavra a ser lida. Se você perder o RIP, o computador "trava" porque ele não sabe mais qual é a próxima ordem a seguir.

Pilha (Stack)

Tuesday, April 21, 2026 1:43 PM

A pilha (ou **Stack**) é uma das estruturas mais genéricas e vitais da computação. No contexto de Sistemas Operacionais e arquitetura x86, ela funciona como uma "memória de rascunho" organizada e extremamente rápida.

Aqui está o que você precisa saber para dominar esse conceito:

1. A Estrutura LIFO (Last-In, First-Out)

A regra de ouro da pilha é: **o último a entrar é o primeiro a sair**. Imagine uma pilha de pratos: para pegar o prato que está no fundo, você precisa primeiro remover todos os que foram colocados em cima.

2. O Papel do RSP (Stack Pointer)

O **RSP** (ou ESP em 32 bits) é o registrador encarregado de "vigiar" a pilha.

- Ele sempre contém o **endereço de memória** do último item que foi colocado lá.
- **Curiosidade técnica:** Na arquitetura x86, a pilha cresce "para baixo". Isso significa que, quando você adiciona um dado, o endereço de memória no RSP **diminui**.

3. Operações Básicas: PUSH e POP

O processador oferece instruções específicas para não precisarmos calcular endereços manualmente:

- **PUSH (Empilhar):**

1. O RSP é subtraído (abre espaço).
2. O valor é copiado para a posição de memória onde o RSP está apontando.

- *Exemplo:* push rax (Salva o valor de RAX na pilha).
- **POP (Desempilhar):**

3. O valor que está no topo (onde o RSP aponta) é copiado para o registrador de destino.
4. O RSP é somado (o espaço é "liberado").

- *Exemplo:* pop rbx (Recupera o valor do topo da pilha e joga no RBX).

4. Por que a Pilha é tão importante?

Ela serve para três propósitos principais que mantêm o computador funcionando:

5. **Chamadas de Funções:** Quando o código chama uma função (CALL), o endereço de retorno (onde o RIP estava) é guardado na pilha para que o programa saiba para onde voltar depois.
6. **Variáveis Locais:** Quando você cria uma variável dentro de uma função em C ou Java, ela vive temporariamente na pilha.
7. **Preservação de Registradores:** Se você precisa usar o RAX para uma conta, mas ele já tem um valor importante, você dá um PUSH RAX, faz sua conta, e depois dá um POP RAX para devolver o valor original.

5. Manipulação como "Memória Comum"

Embora existam o PUSH e o POP, você também pode acessar a pilha diretamente usando endereços, já que ela é apenas uma região da RAM.

- **Exemplo:** mov rax, [rsp + 8] Isso permite que você "espione" o que foi empilhado anteriormente sem precisar remover o item do topo.

  Resumo: A pilha é o mecanismo que permite que funções sejam chamadas umas dentro das outras sem que o processador se perca no caminho de volta. Sem o RSP e a Pilha, não existiriam linguagens de programação modernas!